⚡ Quick Revision Sheet — Read 15 Minutes Before the Viva

1. The 30-second pitch (say it aloud twice now)

> "My FYP is a **Prize Bond Checking App** for Pakistan built with **Flutter and Firebase**. Users instantly check National Savings prize bond results, save a bond portfolio that is **auto-checked on every new draw**, and receive **push notifications when they win**. It has an official **draw schedule**, **PDF download/generation**, a bond **marketplace with WhatsApp contact**, an **admin panel** to publish results and approve users — and the core checking works **fully offline** using a Hive cache."

2. Tech stack — one line each

- **Flutter (Dart)** — one codebase, Android + iOS, hot reload, native performance.
- **GetX** — state management (`.obs` + `Obx`), dependency injection (`Get.put` / `Get.find`), navigation (`Get.to`) — three tools, one package.
- **Firebase Auth** — email/password login; roles admin / normal_user; guest mode allowed.
- **Cloud Firestore** — 5 collections: `customers` , `draws` , `saved_bonds` , `marketplace` , `notifications` .
- **Firebase Storage** — admin's official draw PDFs (`draw_results/{denom}/{drawNo}.pdf`).
- **FCM + Cloud Functions** — push notifications; server matches winners.
- **Hive** — offline cache: 3 boxes (`draws_cache` , `saved_bonds_cache` , `app_settings`).
- **GetStorage** — small key-value data: settings, session, saved bonds, offline sync queue.

3. App flow (draw this if asked)

```
Splash (session+role check)
 ├─ none → Sign In — Guest → Main
 ├─ admin → Admin Dashboard → Create Draw / Manage Users
 └─ user  → Main Screen: [Home | My Bonds | Market | Draws]
Home: quick check + stats + latest draws + Scanner FAB → Scanner returns 6-digit number
Draws: schedule (next-draw countdown) + results + filters + PDF export
Draw Detail: winning numbers + PDF (Download / Generate / Open)
Settings → Profile → Sign Out
```

4. Architecture (say the sentence)

> "MVC with a Service layer using GetX: **Views** are dumb widgets watching reactive state, **Controllers** hold `.obs` state and business logic, **Services** talk to Firestore, Hive, FCM and files, **Models** map documents to Dart objects. Data flows one way: tap → controller → service → data source → Rx update → `Obx` rebuilds only the watching widget."

Startup order (why it matters): Firebase → GetStorage → **Hive before controllers** (DrawController reads cache on init) → local notification channel → FCM background handler → `Get.put` permanent controllers → runApp.

5. The 4 flagship features — internals in 3 lines each

① **Bond check:** select denomination + 6-digit number → `DrawController.checkBond` → loop loaded draws: `denomination` matches AND `winningNumbers.contains(number)` → winner dialog + local notification, or "Not a Winner". Draws come from Firestore online / Hive offline → **works with no internet**.

② **Scanner:** `mobile_scanner` opens back camera → barcode detected → validate regex `^\d{6}$` → stop camera → "Use This Number" → `Get.back(result: number)` fills Home field. **QR chosen over OCR deliberately** — deterministic, fast, reliable on worn paper.

③ **Notifications (2 systems):** - **FCM (server):** login saves the device token to the user's document → admin publishes a draw → Cloud Function `onNewDraw` fires → broadcast push to all active users (chunks of 500) + compares winning numbers with all `saved_bonds` of that denomination → personal "🎉 You Won" push + in-app notification (duplicate-check protected) → invalid tokens cleaned up. - **Local (`flutter_local_notifications`):** channel `prize_bond_channel`; displays messages in foreground/background/terminated (top-level background handler with `@pragma('vm:entry-point')`); tap navigates: `draw_result` →Draws, `winner` →My Bonds.

④ **Offline mode:** `ConnectivityService` (connectivity_plus) exposes reactive `isOnline` → online: Firestore + write-through to Hive; offline: read Hive + banner. Writes (bonds, listings) saved locally first, **queued and synced** when back online. Sentence: "*cache-aside strategy: Firestore is the source of truth, Hive is the offline replica, writes replay on reconnect.*"

6. PDFs — three states

- **Download** (admin uploaded → Firebase Storage, live progress from `bytesTransferred/totalBytes`)
- **Generate** (no admin PDF → `pdf` package builds A4 with 6-column number grid, offline-capable)
- **Open** (already on device — path remembered in Hive settings box). Opened with `open_filex`.

7. Auth flow keywords

signup → Firestore profile with **status: pending** → admin approves (**active**) → login checks: credentials → profile exists → status gate (pending dialog / suspended block) → role matches selector → save FCM token → cache profile → route by role. Errors: friendly message per `FirebaseAuthException` code. On account switch, previous user's local data is **wiped**.

8. Key definitions (one-liners)

- **Widget** — immutable building block of Flutter UI; everything is a widget.
- **`.obs` / Obx** — observable variable / widget that auto-rebuilds when it changes.
- **`Get.put` / `Get.find`** — register / retrieve a controller (dependency injection).
- **IndexedStack** — shows one tab but keeps all alive → no reload on tab switch.
- **Firestore** — NoSQL cloud document database (collections → documents → fields).
- **Cloud Function** — server-side code triggered by events (e.g. new draw document).
- **FCM token** — unique push address of one device; stored per user, auto-refreshed.
- **Hive box** — a fast binary key-value store on the device.
- **Aggregation `count()`** — server counts documents, returns just the number (admin stats).

- **ScreenUtil** — scales sizes from a 400×812 design to any real screen.

9. Numbers to remember

- **8 denominations:** 100, 200, 750, 1500, 7500, 15000, 25000, 40000.
- **6 digits** — bond number format. **500** — FCM multicast chunk size.
- **5** Firestore collections, **3** Hive boxes, **3** Cloud Functions, **4** bottom tabs.
- Schedule: 100→1st monthly; 200 & 750→15th monthly; 1500→1st & 15th; 7500→Feb/May/Aug/Nov; 15000 & 25000→Jan/Apr/Jul/Oct; 40000→Mar/Jun/Sep/Dec.

10. Hard questions — safe answers

- **Why Flutter, not React Native?** Compiles to native code, own rendering engine (no JS bridge), one pixel-perfect UI on both platforms, strong typing.
- **Why GetX, not BLoC/Provider?** Same power, far less boilerplate; state + DI + navigation in one; ideal for a solo developer.
- **Why Firebase?** Full production backend (auth, DB, storage, push, functions) with no server to build or host; free tier; official Flutter SDKs; auto-scaling.
- **Why NoSQL?** Document-shaped data, no joins needed, scales horizontally, flexible schema.
- **Why Cloud Function for winner alerts?** Clients must never hold the FCM server key, and the server runs even when every phone is offline.
- **Security?** Firebase-managed passwords; role+status gates at login and splash; admin callable verifies role **server-side**; per-user data filtered by UID + security rules; local wipe on account switch.
- **Scaling to 1M users?** FCM topics for broadcast, query pagination, composite indexes, App Check, tightened rules — Firestore/FCM themselves auto-scale.
- **Hardest challenge?** End-to-end notifications across foreground/background/terminated states (token lifecycle, background isolate handler, duplicate prevention).
- **Legacy/duplicate files spotted?** "The project was refactored from an early prototype to the current MainScreen architecture; the live path is Splash → MainScreen 4 tabs; old files are kept as reference and scheduled for cleanup — I can show exactly which files are live."
- **Schedule hardcoded?** "Government-fixed dates, so embedded + next-date computed dynamically; Remote Config is future work."

11. Future work (pick 3)

Auto-import results from National Savings source · OCR scanning · Urdu localization + dark mode · marketplace payments/escrow · security-rules hardening + App Check · unit tests.

12. Last tips

- If you don't know an answer: **"I haven't implemented that yet, but my approach would be…"** — never bluff.
- Always tie answers back to a screen: examiners love "I can show you this in the app."
- Open the app on your phone before entering. Charge it. Have the admin login ready.
- Breathe. You built a real, working, multi-service product. Own it.